EXPERIMENT: ENTITY RELATIONSHIP (ER) DIAGRAM DESIGN

1. Aim

To design and understand an Entity Relationship (ER) diagram representing the database structure of a campus event management system with QR-based registration.

2. Objective

- To identify entities involved in the system
- To define attributes of each entity
- To establish relationships between entities
- To understand primary keys and foreign keys
- To model real-world data into a structured database design
- To represent constraints and dependencies between entities

3. Definition of ER Diagram

An Entity Relationship (ER) diagram is a type of diagram used to represent the logical structure of a database.

It shows:

- Entities (tables)
- Attributes (fields/columns)
- Relationships between entities
- Primary Keys (PK) and Foreign Keys (FK)
- Cardinality (one-to-one, one-to-many relationships)

ER diagrams are widely used in database design to visualize how data is stored and connected.

4. Procedure

Step 1: Identify Entities

From the system, the following entities are identified:

- STUDENT
- ADMIN
- EVENT
- REGISTRATION
- QR_CODE
- NOTIFICATION

Step 2: Define Attributes

STUDENT

- student_id (PK)
- name
- email
- department

ADMIN

- admin_id (PK)
- name
- role

EVENT

- event_id (PK)
- title
- description
- date
- time
- location
- category
- admin_id (FK)

REGISTRATION

- registration_id (PK)
- student_id (FK)
- event_id (FK)
- status

QR_CODE

- qr_id (PK)
- code_data
- generated_time
- registration_id (FK)

NOTIFICATION

- notification_id (PK)
- message
- date
- type
- student_id (FK)

Step 3: Identify Relationships

- STUDENT → REGISTRATION
  (A student registers for events)
- EVENT → REGISTRATION
  (An event has multiple registrations)
- REGISTRATION → QR_CODE
  (Each registration generates a QR code)
- ADMIN → EVENT
  (Admin creates and manages events)
- STUDENT → NOTIFICATION
  (Students receive event updates)

Step 4: Define Cardinality

- One STUDENT → Many REGISTRATIONS (1 to many)
- One EVENT → Many REGISTRATIONS (1 to many)
- One REGISTRATION → One QR_CODE (1 to 1)
- One ADMIN → Many EVENTS (1 to many)
- One STUDENT → Many NOTIFICATIONS (1 to many)

Step 5: Assign Keys

Primary Keys (PK):
Uniquely identify each record

- student_id, event_id, registration_id

Foreign Keys (FK):
Establish relationships between tables

- registration.student_id → STUDENT
- registration.event_id → EVENT
- event.admin_id → ADMIN
- qr_code.registration_id → REGISTRATION
- notification.student_id → STUDENT

Step 6: Draw ER Diagram

- Represent entities as rectangles
- Add attributes inside each entity
- Mark PK and FK clearly
- Connect entities using relationship lines
- Label relationships such as:
    - registers
    - creates
    - generates
    - receives

Step 7: Finalize Diagram

- Ensure all entities are properly connected
- Maintain clarity and spacing
- Verify keys and relationships
- Check consistency in naming

5. Notations Used in the Diagram

Notation	Meaning
Rectangle	Entity (table)
Attributes	Columns inside entity
PK	Primary Key
FK	Foreign Key
Line	Relationship between entities
Crow’s Foot	Represents “many”
Single Line	Represents “one”
Relationship Label	Describes interaction

6. Procedure to Create ER Diagram in Draw.io

Step 1: Open https://draw.io
Step 2: Create New Diagram → Blank Diagram
Step 3: Enable *Entity Relation* shapes

Step 4: Add entities:

- STUDENT, ADMIN, EVENT, REGISTRATION, QR_CODE, NOTIFICATION

Step 5: Add attributes and mark PK/FK

Step 6: Draw relationships:

- registers
- creates
- generates
- receives

Step 7: Add cardinality:

- One (|)
- Many (crow's foot)

Step 8: Format:

- Align properly
- Maintain spacing
- Ensure readability

Step 9: Export as PNG/JPEG/PDF

7. Procedure to Upload Diagram in Azure DevOps Wiki

Step 1: Open Azure DevOps → Go to your project
Step 2: Navigate to Wiki section
Step 3: Open/Create page (e.g., *ER Diagram*)

Step 4: Click **Edit**
Step 5: Upload image:

- Drag & drop OR Insert Image

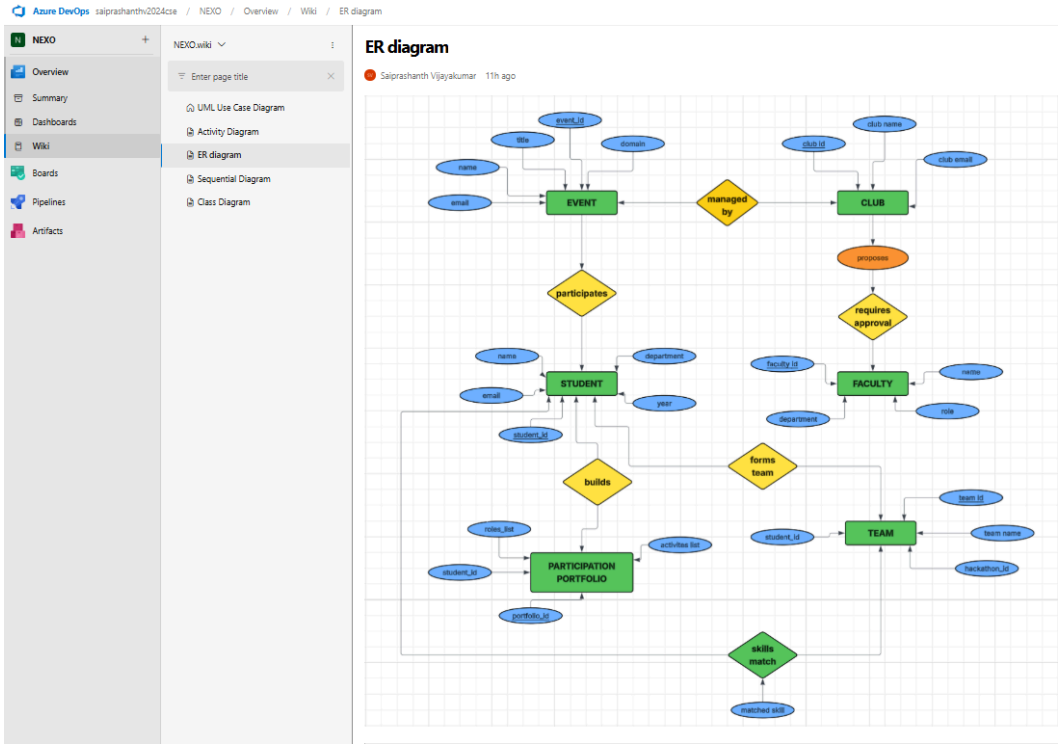
Step 6: Adjust size and alignment

Step 7: Click **Save / Save & Close**
(Add commit message like *"Added ER diagram"*)

Step 8: Verify:

- Ensure diagram is visible
- Check clarity and readability

7. Outcome



8. Result

The ER diagram was successfully created to represent the database structure of the campus event management system. It clearly illustrates the entities, attributes, and relationships, including key constraints such as primary and foreign keys.

The diagram effectively models how data such as events, student registrations, QR codes, and notifications are interconnected. This ensures efficient data organization, seamless registration handling, and quick retrieval of information, supporting a well-structured and scalable system.